

ADAS Intelligence Platform

AI-Powered Engineering Assistant for ADAS Validation, Simulation, Diagnostics, Telemetry Analysis and Safety Evidence

Project Identity

This is not another object detection model and not a claim to build a production self-driving car. It is a production-grade engineering platform that helps ADAS teams analyze logs, generate simulation scenarios, validate behavior, diagnose failures, and create evidence-backed engineering reports.

Document Type	Full Project Specification and Architecture Document
Project Name	ADAS Intelligence Platform (AIP)
Target Roles	AI Systems Engineer, ADAS AI Engineer, LLM/Agent Engineer, Automotive AI Engineer
Target Companies	Automotive OEMs, Tier-1 suppliers, F1/Le Mans engineering teams, robotics companies, AI platform teams
Version	1.0
Date	July 2026

Table of Contents

- 1. Executive Summary
- 2. Problem Statement
- 3. Product Vision and Scope
- 4. Target Users and Use Cases
- 5. ADAS Domain Background
- 6. Functional Requirements
- 7. Non-Functional Requirements
- 8. Detailed System Architecture
- 9. Data Architecture
- 10. Simulation and Scenario Architecture
- 11. AI/LLM Agent Architecture
- 12. RAG and Knowledge Architecture
- 13. Perception and Telemetry Analytics
- 14. Safety, Validation and Evidence
- 15. Security and Governance
- 16. Software and Hardware Specification
- 17. Database Design
- 18. API Specification
- 19. Metrics and Evaluation Criteria
- 20. Testing Strategy
- 21. Deployment and DevOps
- 22. Implementation Roadmap
- 23. GitHub Repository Structure
- 24. Resume and Interview Positioning
- 25. Feature-Level ADAS Specifications
- 26. Dashboard and UI Specification
- 27. Data Quality and Validation Rules
- 28. Motorsport / F1 Extension
- 29. Risk Register
- 30. References

1. Executive Summary

The ADAS Intelligence Platform (AIP) is a production-grade AI system designed to support development, testing, validation and diagnostics of Advanced Driver Assistance Systems. Instead of building a single perception model, the platform focuses on the high-value engineering problems around ADAS: understanding sensor logs, generating test scenarios, validating system behavior, diagnosing failures, explaining root causes, and creating safety evidence.

The project combines LLM orchestration, multi-agent workflows, retrieval-augmented generation, time-series analytics, simulation control, perception evaluation, telemetry analysis and safety-aware reporting. The goal is to demonstrate that the developer can build AI infrastructure for complex, safety-critical engineering domains such as automotive, motorsport, robotics and autonomous systems.

Flagship Value
AIP proves capability in both frontier AI systems and automotive engineering. It shows that the developer can build tools that engineers would actually use, rather than only training another YOLO or CNN model.

1.1 Core Product Outcome

- 1 An engineer uploads ADAS logs, simulation runs, video, CAN data or technical documents.
- 2 The platform aligns and indexes data across time, sensors and documents.
- 3 AI agents inspect the incident, retrieve relevant engineering knowledge and run simulations or analytics.
- 4 The system generates evidence-backed root cause reports, safety metrics and improvement recommendations.
- 5 Every output is traceable to logs, documents, metrics or simulation evidence.

1.2 Why This Is Better Than a Normal Student Project

Normal Student Project	ADAS Intelligence Platform
Trains one model on one dataset	Builds a complete engineering platform around data, simulation, agents and validation
Mostly academic metrics only	Uses production KPIs: latency, cost, false negatives, scenario coverage, evidence quality
Hard to connect to industry workflow	Directly maps to ADAS validation, diagnostics, simulation and safety workflows
Limited system design	Includes API gateway, data pipelines, agents, databases, dashboards, deployment and monitoring

2. Problem Statement

Modern vehicles are increasingly software-defined. ADAS features such as automatic emergency braking, adaptive cruise control, lane keeping assist, blind spot monitoring, driver monitoring and traffic sign recognition depend on complex interactions between sensors, perception models, planning logic, control systems, calibration, maps, driver behavior and environment conditions. When an ADAS feature fails or behaves unexpectedly, the root cause is rarely visible from one file or one model output.

Engineers usually need to inspect sensor logs, CAN signals, diagnostic trouble codes, camera videos, radar or lidar outputs, software versions, requirement documents, calibration settings and simulation traces. This creates a bottleneck: debugging and validation become slow, fragmented and manual.

2.1 Major Industry Pain Points

Pain Point	Why It Matters	AIP Response
Large multi-modal logs	Camera, radar, lidar, GPS, IMU and CAN data are hard to correlate manually.	Time-synchronized ingestion and event segmentation.
Difficult root cause analysis	Failures may be caused by sensor noise, calibration, perception thresholds, planner state or requirements mismatch.	Multi-agent diagnostics with evidence-backed hypotheses.
Scenario coverage gaps	Rare edge cases cause many ADAS failures.	Scenario generation, mutation and regression banks.
Unclear safety evidence	Teams need traceability between requirements, tests, logs and results.	Requirements-to-evidence linking and automated reports.
Slow validation cycles	Manual replay and analysis slows release.	Automated simulation, scoring, dashboards and release gates.
LLM risk in engineering	LLMs can hallucinate or make unsupported claims.	Grounding, verification, citations and human approval gates.

2.2 Project Mission

Mission Statement

Build an AI platform that turns raw ADAS data and engineering knowledge into traceable, measurable and explainable validation intelligence.

3. Product Vision and Scope

AIP should feel like an engineering workbench for ADAS teams. It should not claim to replace certified engineering workflows. Instead, it should accelerate analysis, improve traceability, improve scenario coverage and provide structured evidence that engineers can review.

3.1 Product Vision

To create an AI-powered ADAS engineering platform that can understand vehicle logs, simulation scenarios, sensor outputs and technical documentation, then assist engineers in diagnosing failures, generating tests and validating behavior.

3.2 In Scope

- ADAS log ingestion and indexing: CAN logs, CSV telemetry, ROS bags, camera/video files, DBC files and simulation outputs.
- Automotive RAG over manuals, requirements, ADAS specifications, DBC signal definitions, test cases and known issues.
- Multi-agent engineering workflows for diagnostics, simulation planning, perception review, telemetry analysis and report generation.
- CARLA-based simulation integration for scenario execution and replay.
- OpenSCENARIO and OpenDRIVE-aware scenario management for validation workflows.
- Metrics dashboard for ADAS performance, LLM performance, scenario coverage, cost and latency.
- Safety evidence generation with citations to logs, requirements and simulation outputs.
- Production-grade deployment using Docker, optional Kubernetes, monitoring and CI/CD.

3.3 Out of Scope for Version 1

- A production-certified autonomous driving stack.
- Real road deployment without certified hardware, compliance, safety case and regulatory approval.

- Replacing professional safety engineers or functional safety teams.
- Training foundation models from scratch.
- Access to proprietary OEM datasets unless provided by partner or employer.

3.4 Product Modules

Module	Purpose
Data Ingestion Layer	Import, validate, synchronize and segment ADAS data.
Automotive Knowledge Layer	Index requirements, manuals, DBC files, design docs and test specs.
Agent Orchestration Layer	Coordinate diagnostic, simulation, perception and safety agents.
Simulation Layer	Generate, execute and score scenarios in CARLA/ROS 2.
Analytics Layer	Compute ADAS metrics, anomaly scores, perception performance and telemetry KPIs.
Verification Layer	Check LLM claims against evidence and reject unsupported statements.
Dashboard Layer	Visualize events, scenarios, metrics, reports and model behavior.
DevOps Layer	Deploy, monitor, test and operate the platform.

4. Target Users and Use Cases

Persona	Needs	Platform Features
ADAS Validation Engineer	Find why a scenario failed and whether the feature meets requirement thresholds.	Scenario replay, metrics, evidence reports, requirements tracing.
Perception Engineer	Analyze missed detections, false positives, tracking instability and confidence drops.	Perception validation, slice analysis, video/frame review, model comparison.
Controls/Planning Engineer	Investigate late braking, lane deviation, unsafe following distance and comfort issues.	Telemetry analytics, TTC calculations, state-machine traces, simulation replay.
Test Engineer	Create many edge-case scenarios and regression suites.	LLM scenario authoring, OpenSCENARIO generation, parameter mutation, scenario bank.
Safety Engineer	Collect evidence for safety review and release decisions.	Traceability matrix, safety evidence reports, human approval queue.
Program Manager	Understand readiness, risk and release blockers.	Dashboards, KPIs, issue summaries, trend charts.

4.1 Example Use Cases

- 1 Ask: "Why did lane keeping fail in this rainy curve scenario?" The system retrieves the requirement, analyzes lane offset and steering commands, checks camera confidence and generates a root cause report.
- 2 Ask: "Generate 50 cut-in scenarios at 40-80 km/h with different weather and visibility conditions." The scenario agent creates parameterized scenarios and queues them for simulation.
- 3 Ask: "Compare AEB performance between model v2 and model v3." The platform computes TTC, collision rate, braking latency and false positive rate across a scenario suite.
- 4 Ask: "Show all night-time pedestrian false negatives in the last validation batch." The system filters events, groups failures and shows evidence frames.
- 5 Ask: "Create a release evidence report for lane keeping assist." The platform connects requirements, test cases, scenario results and known limitations.

5. ADAS Domain Background

ADAS is a collection of vehicle systems that assist the driver using sensing, perception, decision-making and actuation. Common features include adaptive cruise control, lane keeping assist, automatic emergency braking, blind spot monitoring, traffic sign recognition, driver monitoring and parking assistance.

5.1 ADAS Pipeline

Stage	Typical Data	Examples
Sensing	Camera, radar, lidar, ultrasonic, GPS, IMU, wheel speed, steering angle	Raw images, point clouds, radar tracks, vehicle speed
Perception	Detected objects, lanes, signs, freespace, tracks	Pedestrian boxes, lane boundaries, object velocity
Prediction	Predicted object motion and risk	Cut-in risk, pedestrian crossing probability
Planning / Decision	ADAS state machine and maneuver decision	AEB trigger, ACC target selection, lane keeping active
Control	Steering, braking, acceleration commands	Brake pressure, steering torque request
Validation	Metrics, pass/fail, safety evidence	TTC, lane offset, collision count, false positives

5.2 Safety Context

Automotive safety work commonly distinguishes between hazards caused by faults in electrical/electronic systems and hazards related to limitations of intended functionality. ISO 26262 is commonly associated with functional safety of automotive E/E systems, while SOTIF/ISO 21448 focuses on risks from intended functionality limitations and reasonably foreseeable misuse. AIP is not a compliance tool, but it is designed to organize evidence in a way that resembles real validation workflows.

5.3 Key External Standards and Tools to Align With

Area	Standard / Tool	How AIP Uses It
Simulation	CARLA	Open-source simulator for development, training and validation of autonomous driving systems.
Scenario Description	ASAM OpenSCENARIO	Reusable scenario descriptions for ADAS/AV V&V; and scenario-based testing.
Road Network	ASAM OpenDRIVE	Road network descriptions that can be exchanged between simulators.
Robotics Middleware	ROS 2	Message passing, topics, services and actions for simulation and robot/vehicle components.
Automotive Software	AUTOSAR Adaptive	Useful reference model for high-performance automotive software architecture.
Datasets	nuScenes / nuPlan / KITTI style datasets	Public data for perception, tracking, prediction and planning experiments.
LLM Security	OWASP LLM Top 10	Threat model for prompt injection, insecure output, data disclosure and excessive agency.

6. Detailed System Architecture

The architecture is separated into seven layers: client layer, API/control layer, data layer, AI/agent layer, simulation layer, persistence layer and operations layer. This separation makes the system easier to scale, test and explain in interviews.

6.1 System Context Diagram

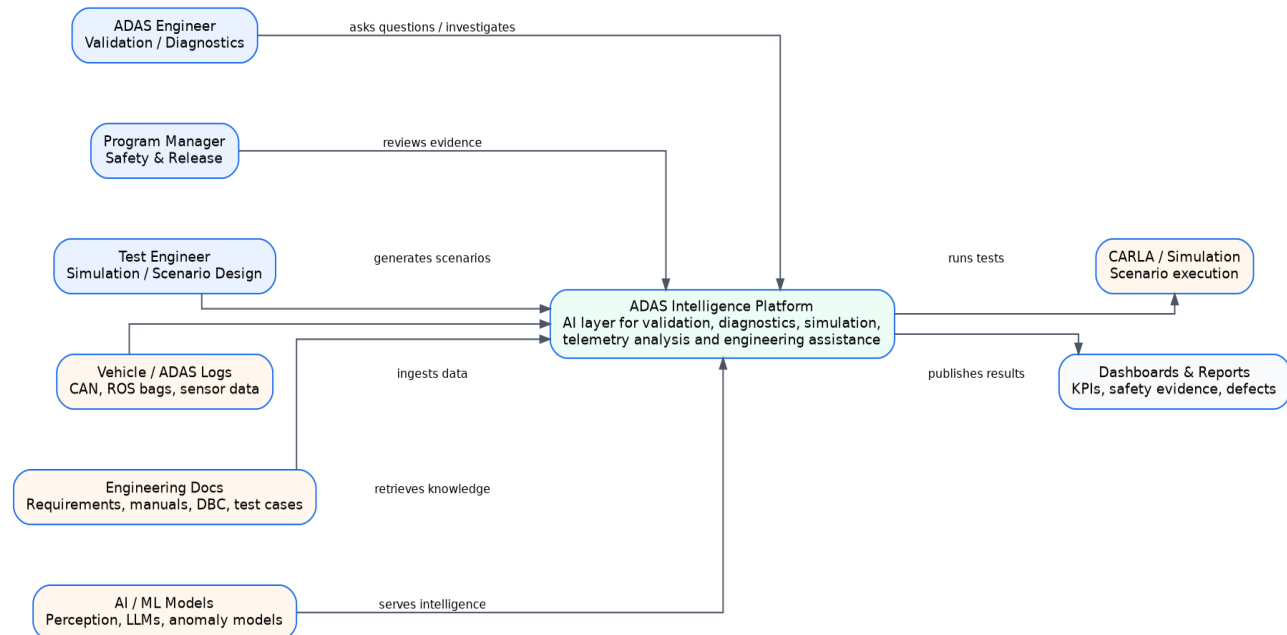


Figure 1: AIP connects engineers, documents, logs, models, simulators and reports into one ADAS intelligence workflow.

6.2 Container Architecture

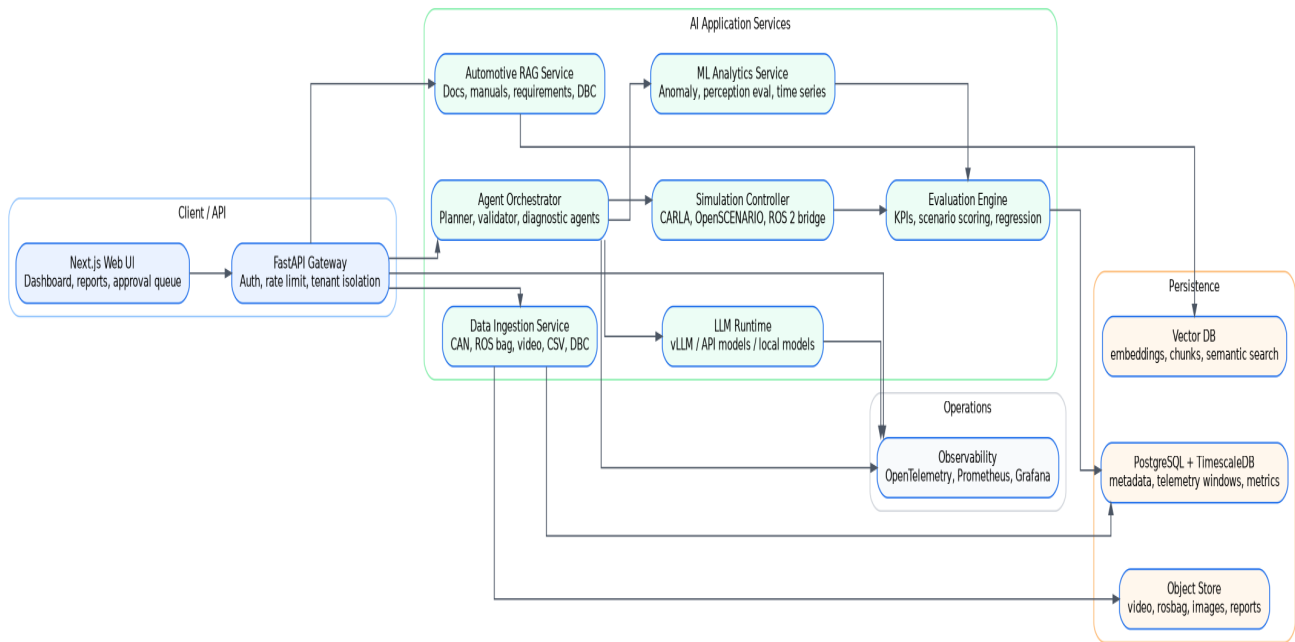


Figure 2: Production-style container architecture with API, agents, ingestion, simulation, LLM serving, analytics and observability.

6.3 Architectural Principles

Principle	Meaning
Evidence-first AI	Every engineering claim should point to logs, metrics, documents or simulation output.

Principle	Meaning
Human-supervised autonomy	AI can investigate and recommend, but risky claims or actions require human approval.
Simulation before deployment	New hypotheses and scenarios are validated in simulation or offline analysis first.
Traceability by design	Requirements, scenarios, logs, metrics and reports are connected through IDs and metadata.
Modular AI	LLMs, perception models, anomaly detectors and scenario generators are swappable components.
Production observability	Latency, cost, accuracy, tool calls, failures and safety gates are logged from day one.

7. Functional Requirements

ID	Requirement	Priority
FR-01	User authentication and project workspace management.	Must have
FR-02	Upload and manage ADAS data files: CSV, video, images, DBC, ROS bag metadata, logs and documents.	Must have
FR-03	Parse and index vehicle telemetry with timestamp alignment.	Must have
FR-04	Parse DBC signal definitions and map raw signals into human-readable features.	Should have
FR-05	RAG over engineering documents, requirements, manuals and issue notes.	Must have
FR-06	Agent workflow for root cause analysis from an incident.	Must have
FR-07	Simulation scenario generation using natural language intent and parameter ranges.	Should have
FR-08	CARLA scenario execution and result collection.	Should have
FR-09	ADAS metrics computation: TTC, lane offset, collision, braking latency, false positive/negative rates.	Must have
FR-10	Perception model evaluation against ground truth or simulation labels.	Should have
FR-11	Evidence-backed report generation with confidence and citations.	Must have
FR-12	Human approval queue for high-risk claims and tool actions.	Should have
FR-13	Model/prompt evaluation dashboard for LLM and agent performance.	Must have
FR-14	Export reports as PDF/HTML/Markdown.	Should have
FR-15	Admin dashboard for datasets, scenarios, incidents, metrics and users.	Must have

8. Non-Functional Requirements

ID	Category	Requirement
NFR-01	Reliability	The platform should not lose uploaded files or generated evidence; all background jobs must be retryable.
NFR-02	Traceability	Every report must include source references to logs, documents, scenarios or metrics.
NFR-03	Security	RBAC, API keys, audit logging, secrets management and prompt injection protection are required.
NFR-04	Scalability	Ingestion, simulation, model serving and agent execution should scale independently.
NFR-05	Latency	Interactive queries should target useful response under 10 seconds for small projects; long jobs run asynchronously.

ID	Category	Requirement
NFR-06	Cost Control	Use local models, caching and model routing to avoid unnecessary expensive LLM calls.
NFR-07	Maintainability	Services should be modular, typed, tested and documented.
NFR-08	Reproducibility	Scenario versions, model versions, dataset versions and configuration must be recorded.
NFR-09	Observability	Every request, agent step, tool call and scenario run should emit metrics and traces.
NFR-10	Safety Consciousness	The system must clearly state that generated advice is engineering assistance, not safety certification.

9. Data Architecture

ADAS data is high-volume, multi-modal and time-dependent. AIP must treat data management as a first-class problem, not just an upload button. The data architecture includes file ingestion, schema validation, timestamp synchronization, event segmentation, feature extraction, indexing and storage.

9.1 Data Ingestion Pipeline

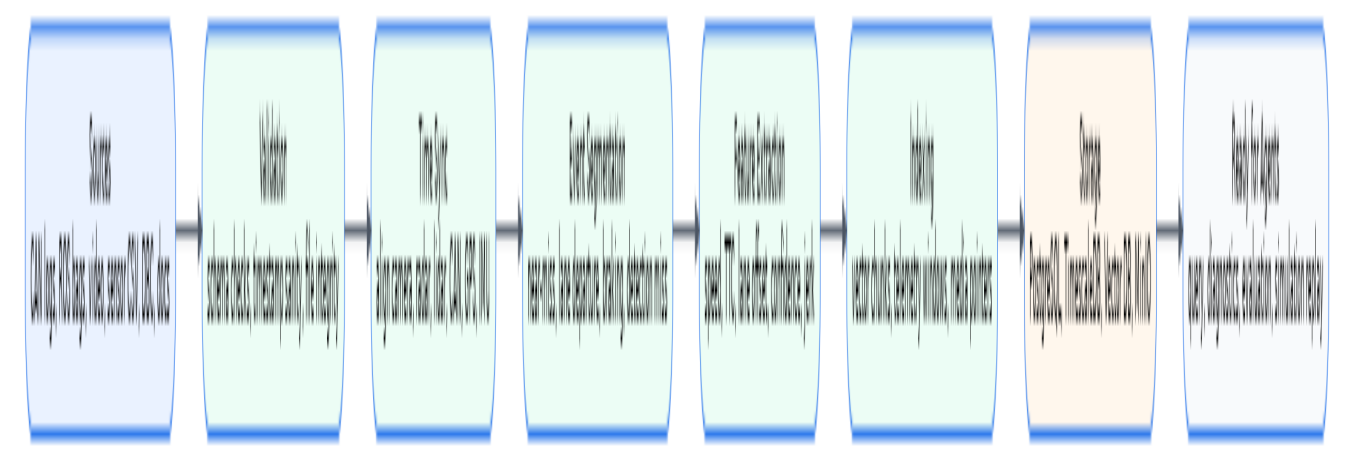


Figure 3: Raw ADAS data is converted into synchronized, searchable and analyzable evidence.

9.2 Supported Data Types

Data Type	Format Examples	Use in Platform
Vehicle telemetry	CSV, MDF-converted CSV, Parquet	Speed, steering, brake, throttle, yaw rate, ACC state, AEB state.
CAN definitions	DBC	Decode raw CAN signals and create human-readable variables.
Video / Images	MP4, AVI, JPG, PNG	Visual incident review and perception model validation.
ROS data	ROS bag metadata or extracted topics	Simulation/robotics style sensor and control signals.
Simulation data	CARLA logs, JSON, CSV, sensor captures	Scenario replay, scoring and regression testing.
Documents	PDF, Markdown, DOCX, TXT	Requirements, test plans, manuals, design notes, issue history.
Annotations	COCO, KITTI, nuScenes-style labels	Perception evaluation and failure slicing.

9.3 Time Synchronization Strategy

A core challenge is aligning heterogeneous signals. The system should normalize all streams into a common time base, store source timestamps and compute derived windows around events.

- Convert all timestamps to a canonical UTC or monotonic simulation time.
- Store original timestamp, normalized timestamp and source clock domain.
- Align data by nearest-neighbor, interpolation or fixed time window based on signal type.
- Build event windows such as T-5s to T+5s around AEB trigger, lane departure or collision.
- Preserve uncertainty: never hide missing, delayed or interpolated data.

9.4 Feature Engineering Examples

Feature	Formula / Meaning	Use
Time-to-collision (TTC)	$\text{relative_distance} / \text{relative_speed}$ when closing	AEB and collision risk analysis.
Lane center offset	vehicle lateral position relative to lane center	Lane keeping performance.
Braking latency	time between risk detection and brake command	AEB responsiveness.
Jerk	change of acceleration over time	Comfort and control smoothness.
Object confidence drop	decrease in detection confidence across frames	Perception stability.
Sensor disagreement	difference between camera/radar/lidar estimate	Fusion anomaly detection.
Scenario severity	weighted score using speed, TTC, collision, occlusion, weather	Prioritize failures.

10. Simulation and Scenario Architecture

Simulation is the safest and most practical way for a student or fresher to build an automotive AI platform without access to proprietary vehicles. AIP should integrate simulation not as a gimmick, but as a validation engine. CARLA, OpenSCENARIO and OpenDRIVE are strong choices because they map to real ADAS/AV engineering workflows.

10.1 Simulation Validation Loop



Figure 4: Requirements are converted into scenarios, executed in simulation, monitored, scored and turned into evidence.

10.2 Scenario Generation Architecture

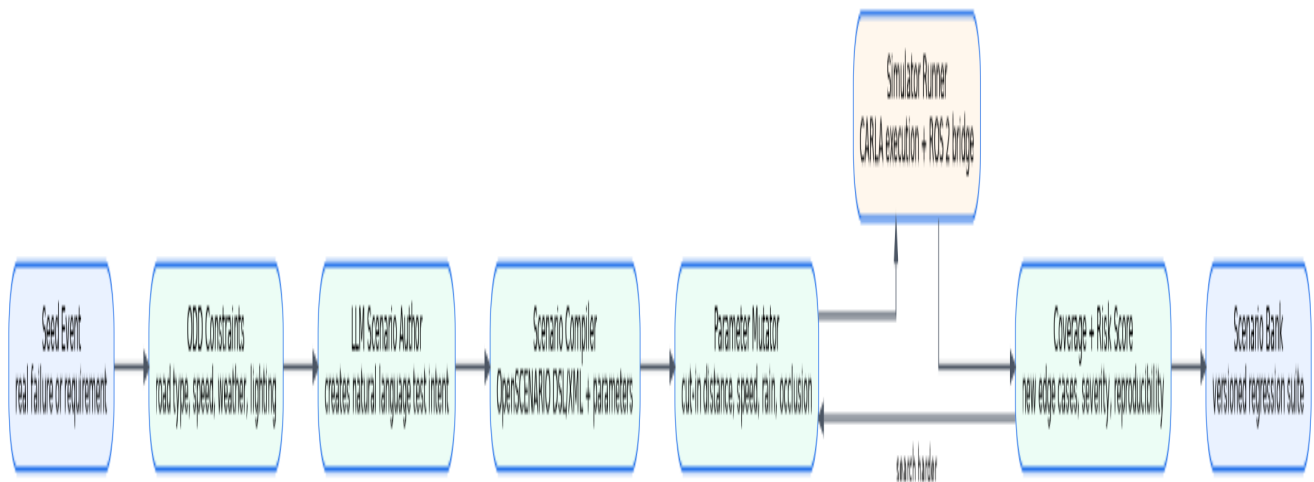


Figure 5: The platform creates parameterized scenarios, mutates them and stores reproducible regression cases.

10.3 Scenario Types to Implement First

Scenario	ADAS Feature	Important Metrics
Lead vehicle sudden braking	AEB / ACC	TTC, brake latency, collision, min distance.
Vehicle cut-in	ACC / AEB	Target selection accuracy, braking smoothness, TTC.
Lane curve with low visibility	Lane keeping assist	Lane offset, lane invasion, steering stability.
Pedestrian crossing with occlusion	AEB / perception	Detection distance, false negative rate, braking latency.
Traffic sign under poor lighting	Traffic sign recognition	Recognition accuracy, confidence, missed signs.
Ghost object / radar false positive	Fusion / AEB	False positive rate, unnecessary braking.

10.4 CARLA Integration Plan

- 1 Run CARLA server locally or on a GPU machine.
- 2 Use Python client to spawn ego vehicle, NPC vehicles, pedestrians and sensors.
- 3 Use OpenDRIVE maps where possible or CARLA built-in towns for development.
- 4 Represent scenarios as JSON first, then evolve to OpenSCENARIO support.
- 5 Record sensor data, collision events, lane invasions, actor positions and control commands.
- 6 Score each scenario using AIP evaluation metrics.
- 7 Store scenario configuration, seed, map, weather, model version and result for reproducibility.

10.5 OpenSCENARIO / OpenDRIVE Strategy

Version 1 can use a simple internal JSON schema. Version 2 should export and import OpenSCENARIO-like descriptions. OpenDRIVE support should be used for road network representation when integrating with standardized simulation maps.

```

{
  "scenario_id": "aeb_cut_in_rain_001",
  "map": "Town05",
  "weather": {"rain": 70, "fog": 20, "sun_altitude": -5},
  "ego": {"speed_kmh": 60, "lane": 2},
  "actors": [
    {"type": "vehicle", "behavior": "cut_in", "initial_distance_m": 25, "relative_speed_kmh": -10}
  ],
  "success_criteria": {
    "collision": false,
    "min_ttc_s": 1.2,
    "max_deceleration_mps2": 6.0
  }
}

```

}
}

11. AI / LLM Agent Architecture

The LLM part of AIP should not be a simple chatbot. It should be an agentic engineering system where each agent has a clear responsibility, tools, permissions and evaluation criteria.

11.1 Agent Orchestration Diagram

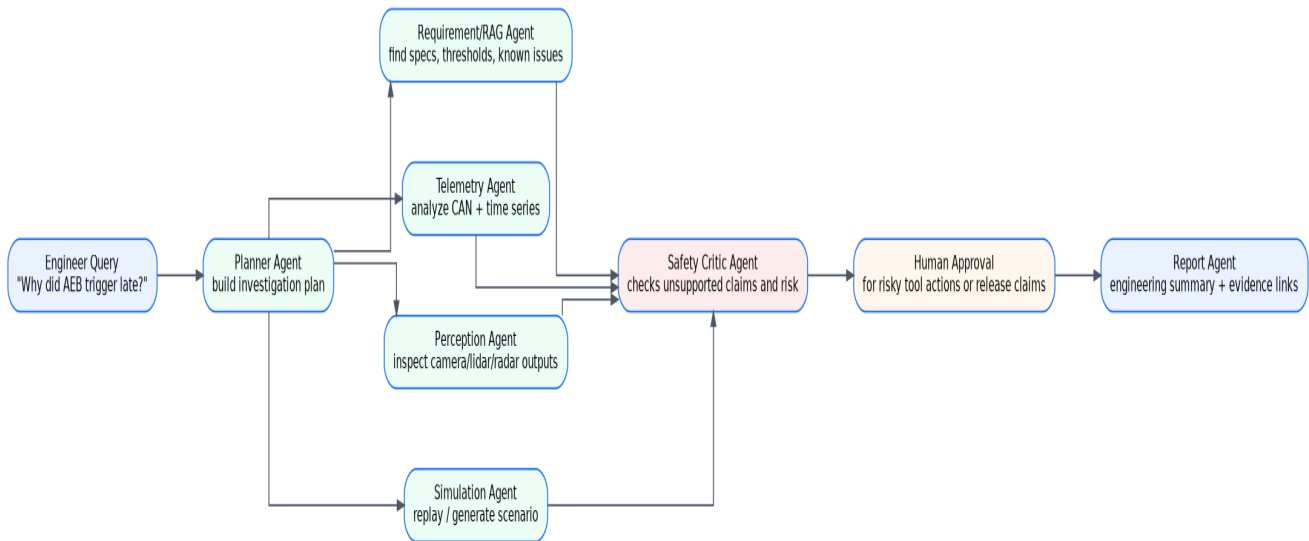


Figure 6: Specialized agents collaborate to investigate ADAS incidents and produce evidence-backed reports.

11.2 Agent Roles

Agent	Responsibility	Tools
Planner Agent	Break user query into investigation steps and choose required tools.	Task graph, policy engine, memory.
Telemetry Agent	Analyze time-series, CAN signals, speed, brake, steering and ADAS state transitions.	Pandas, TimescaleDB, plotting, signal decoders.
Perception Agent	Inspect object/lane detection output, ground truth, video frames and confidence changes.	OpenCV, FiftyOne, PyTorch metrics, object store.
Simulation Agent	Generate or replay CARLA scenarios and collect results.	CARLA API, ROS 2 bridge, scenario runner.
RAG Agent	Retrieve relevant requirements, manuals, DBC definitions and issue history.	Vector DB, keyword search, document parser.
Safety Critic Agent	Check if claims are supported and whether disclaimers or human approval are required.	Evidence verifier, policies, OWASP guardrails.
Report Agent	Generate final engineering report with metrics, charts, source references and recommendations.	PDF/HTML generator, templates.

11.3 Agent Execution State Machine

- 1 Receive query and classify intent: diagnostics, simulation, metrics, knowledge lookup, report generation.
- 2 Create plan with required evidence sources and tools.
- 3 Execute tool calls under policy constraints.
- 4 Store intermediate artifacts, plots and observations.
- 5 Run verifier to reject unsupported claims.
- 6 Ask for human approval if risky action or safety-critical conclusion is involved.
- 7 Generate final report with confidence and evidence links.

11.4 Prompting Strategy

Prompts should be structured, role-specific and evidence-focused. AIP should avoid vague free-form prompts for critical workflows. Each agent should receive task instructions, allowed tools, evidence constraints and output schema.

```
SYSTEM: You are the ADAS Telemetry Agent. Use only provided telemetry windows and decoded signals.
TASK: Identify whether late braking was caused by perception delay, planner state, or control delay.
RULES:
- Do not claim a cause unless supported by timestamped evidence.
- Return uncertainty when evidence is missing.
- Output JSON with observations, evidence_ids, confidence, and next_tests.
OUTPUT_SCHEMA:
{
  "observations": [],
  "hypotheses": [{"cause": "", "evidence_ids": [], "confidence": 0.0}],
  "missing_evidence": [],
  "recommended_next_tests": []
}
```

11.5 Model Strategy

Task	Recommended Model Type	Reason
General reasoning and report writing	Strong API LLM or high-quality local instruct model	Needs reasoning and language quality.
Automotive RAG answering	Medium LLM + good retrieval	Grounded answers matter more than raw model size.
Signal classification/anomaly	Classical ML or small neural network	Fast and explainable for time-series patterns.
Code/log parsing	Code-specialized LLM	Better at scripts, JSON, traces and parsers.
Perception validation	PyTorch/OpenCV metrics, not only LLM	LLM should explain results, not replace metric computation.
Offline low-cost mode	Local LLM through vLLM/Ollama	Useful for demo and privacy.

12. RAG and Automotive Knowledge Architecture

Automotive RAG is different from generic document Q&A.; The platform must understand requirements, DBC signal meanings, test cases, incident reports, calibration notes and versioned documentation. It should retrieve the right evidence and cite it in engineering reports.

12.1 Knowledge Sources

- Requirement documents: feature requirements, acceptance criteria, safety notes.
- Test specifications: scenario descriptions, pass/fail thresholds, regression plans.
- DBC files: CAN signal names, scaling, units and descriptions.
- Service/diagnostic documents: diagnostic trouble codes, failure conditions, recovery procedures.
- Software release notes: model versions, calibration changes, known issues.
- Incident reports: previous failures and root cause summaries.
- Simulation scenario documentation: maps, ODD, parameter ranges and expected behavior.

12.2 RAG Pipeline

Step	Description
Parse	Extract text, tables, headings, metadata and source version.
Chunk	Chunk by semantic section: requirement, signal definition, procedure, test case.

Step	Description
Embed	Generate embeddings using local or API embedding model.
Index	Store chunks in vector DB and metadata in PostgreSQL.
Retrieve	Use hybrid search: vector similarity + keyword + metadata filters.
Rerank	Rerank chunks based on task, source trust and recency.
Answer	Generate answer with strict citations and uncertainty handling.
Verify	Check that every claim is supported by retrieved chunks or computed metrics.

12.3 Metadata Schema for Knowledge Chunks

```

document_id: string
chunk_id: string
source_type: requirement | dbc | manual | test_spec | issue | release_note
vehicle_platform: string
feature: AEB | ACC | LKA | TSR | BSD | DMS
version: string
valid_from: date
access_level: public | internal | restricted
text: string
embedding_id: string
related_signal_names: list[string]
related_scenario_ids: list[string]

```


13. Perception and Telemetry Analytics

13.1 Perception Validation Pipeline



Figure 7: Perception outputs are compared against labels or simulation ground truth and analyzed by scenario slices.

13.2 LLM Diagnostics Workflow

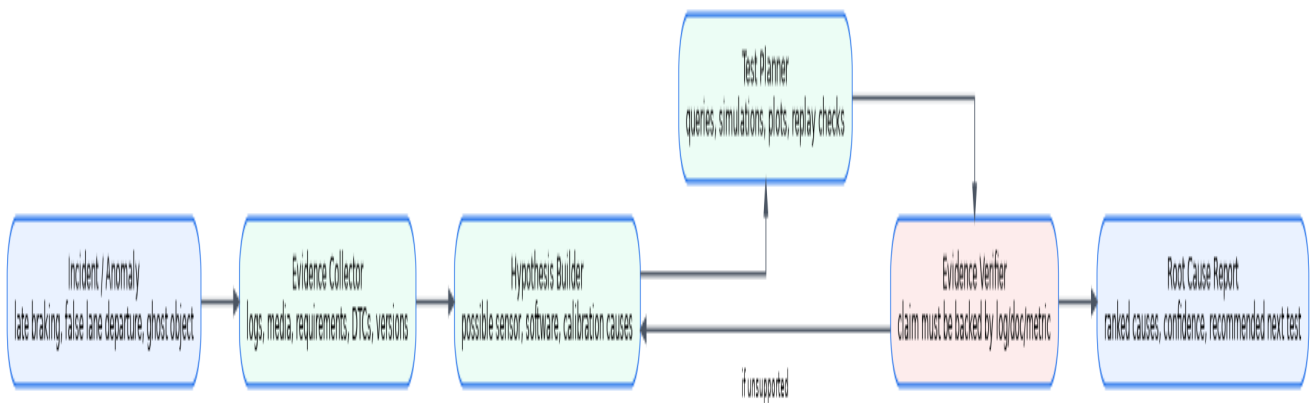


Figure 8: LLM agents generate hypotheses only after collecting evidence and running checks.

13.3 ADAS Metrics Library

Metric	Feature Area	Meaning
Collision rate	AEB / Simulation	Percentage of scenarios ending in collision.
Minimum TTC	AEB / ACC	Smallest time-to-collision during event.
Braking latency	AEB	Time from risk threshold crossing to brake command.
False positive rate	AEB / Perception	Unnecessary warning/braking/detection.
False negative rate	Perception / AEB	Missed object or missed risk event.
Lane invasion count	Lane keeping	Number of lane boundary violations.
Mean lane center offset	Lane keeping	Average lateral deviation from lane center.
Object detection mAP	Perception	Object detection accuracy across classes.
Tracking ID switches	Perception tracking	Track identity instability.
Comfort jerk score	Control	Smoothness of braking/acceleration/steering.
Scenario coverage	Validation	Coverage across ODD dimensions such as speed, weather, lighting and road type.
Evidence support rate	LLM reliability	Percentage of generated claims linked to source evidence.

13.4 Failure Taxonomy

Failure Class	Examples	Likely Evidence
Sensor limitation	Low light, rain, glare, dirty camera, radar ghost	Confidence drops, sensor disagreement, weather metadata.
Perception error	Missed pedestrian, wrong lane, false object	Ground truth mismatch, false negative/positive metrics.
Fusion error	Camera and radar disagree, wrong track association	Track mismatch, delayed signal, ID switch.
Planner/logic error	Late decision, wrong target, state transition issue	ADAS state machine, planner logs, requirement mismatch.
Control issue	Harsh braking, poor steering, oscillation	Actuation commands, acceleration, jerk, path tracking.
Calibration/config issue	Threshold changed, model version mismatch	Config files, release notes, calibration metadata.
Data/logging issue	Missing topic, timestamp drift, corrupted file	Ingestion validation errors and data quality reports.

14. Safety, Validation and Evidence

AIP must be safety-conscious by design. It should never claim that an ADAS feature is safe based only on an LLM answer. The platform should support evidence collection, traceability and engineering review.

14.1 Safety Validation V-Model View



Figure 9: AIP supports traceability from requirement to validation evidence.

14.2 Evidence Model

Evidence Type	Examples	Used For
Requirement evidence	Requirement ID, threshold, expected behavior	Explain what should have happened.
Telemetry evidence	Speed, TTC, brake command, lane offset	Prove what actually happened.
Perception evidence	Detection confidence, bounding box, missed object	Prove what model perceived.
Simulation evidence	Scenario config, seed, run logs, collisions	Reproduce and test behavior.
Version evidence	Model version, calibration, software commit	Identify release-specific failures.

Evidence Type	Examples	Used For
Human review evidence	Approval, comments, decision record	Support release and audit workflows.

14.3 Report Confidence Levels

Confidence	Definition
High	Multiple independent evidence sources support the cause and no major contradictory evidence exists.
Medium	Evidence supports a likely cause, but some data is missing or alternative causes remain.
Low	Evidence is incomplete; report should recommend next tests rather than assert a cause.
Blocked	Insufficient data quality, missing files or unsupported user request.

14.4 Safety Disclaimers to Build Into Product

- AIP is an engineering assistance and validation intelligence platform, not a certified safety tool.
- All safety-critical conclusions require review by qualified engineers.
- LLM-generated outputs must be evidence-checked before use in release decisions.
- Simulation results do not automatically transfer to real road safety without validation.

15. Security and Governance

The platform handles sensitive engineering data, potentially including vehicle logs, proprietary requirements and AI model behavior. Security must be designed from the beginning.

15.1 Security Architecture

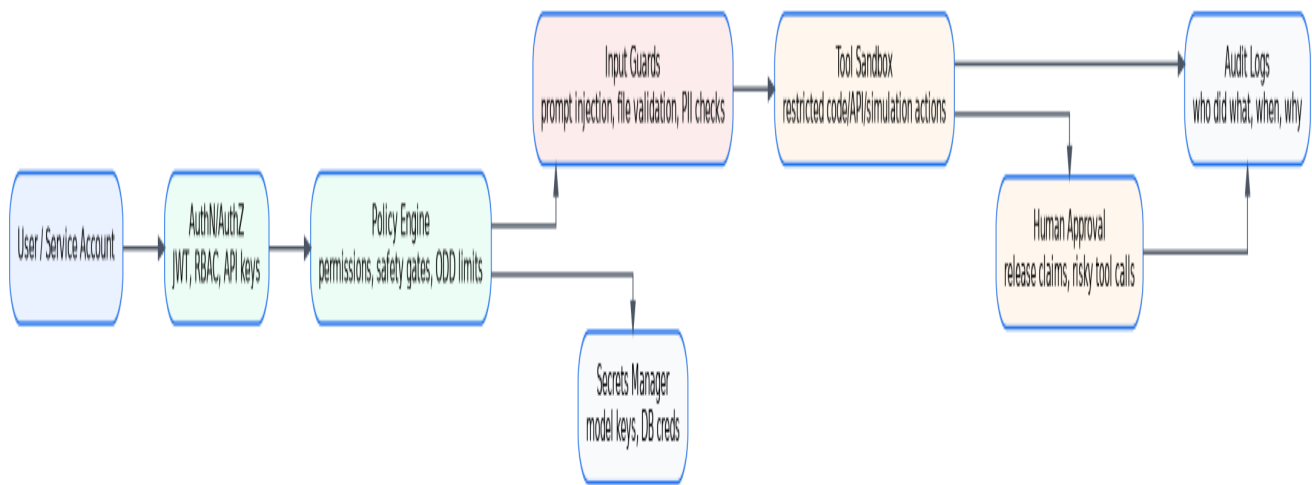


Figure 10: Security layer for authentication, policy enforcement, tool sandboxing and auditability.

15.2 Threats and Controls

Threat	Risk	Control
Prompt injection	User or document tries to override agent rules.	Input guards, tool allowlists, instruction hierarchy, output verification.
Sensitive data leakage	Reports expose restricted logs or documents.	RBAC, access filters, redaction and audit logs.
Excessive agency	Agent executes risky tool action automatically.	Human approval, sandboxing, scoped tokens.
Model hallucination	Unsupported engineering conclusion.	Claim verification, citations, confidence scoring.
Data poisoning	Bad document or labels affect retrieval/evaluation.	Source trust scoring, review workflow, versioning.
Simulation abuse	Expensive runaway scenario generation.	Rate limits, job quotas, budget controls.
Credential exposure	API keys or DB secrets leaked.	Secrets manager, environment isolation, no secrets in logs.

15.3 Governance Requirements

- Every dataset, model, prompt and scenario must have version metadata.
- All report-generating workflows should be reproducible using stored run IDs.
- Human approvals should include reviewer, timestamp, decision and reason.
- Restricted documents should never be retrieved for users without permission.
- Generated reports should include uncertainty and limitations section.

16. Software and Hardware Specification

16.1 Recommended Tech Stack

Layer	Recommended Tools	Reason
Frontend	Next.js, TypeScript, Tailwind CSS, ShadCN UI, Recharts	Professional dashboard, charts and report interface.
Backend API	Python 3.11+, FastAPI, Pydantic, SQLAlchemy	Fast development, typed APIs, ML ecosystem compatibility.
Background Jobs	Celery/RQ/Dramatiq + Redis	Long-running ingestion, simulations and reports.
Database	PostgreSQL + TimescaleDB extension	Metadata plus time-series telemetry windows.
Vector Search	pgvector or Qdrant	Semantic search over documents and incidents.
Object Storage	MinIO/S3	Videos, ROS bags, simulation frames, reports.
LLM Serving	vLLM, Ollama optional, OpenAI-compatible API abstraction	Local and API model flexibility.
Agents	LangGraph or custom state-machine orchestration	Controlled multi-step workflows with traceability.
Simulation	CARLA, Python API, ROS 2 bridge	ADAS/AV scenario simulation and sensor generation.
Robotics Middleware	ROS 2	Useful for sensor topics, simulation bridge and modular components.
Perception/Analytics	PyTorch, OpenCV, NumPy, Pandas, SciPy, scikit-learn	Model evaluation, telemetry processing and anomaly detection.
Annotation/Visualization	FiftyOne, CVAT optional, Plotly/Recharts	Dataset inspection and failure visualization.
DevOps	Docker, Docker Compose, GitHub Actions, Kubernetes optional	Deployment, testing and production-like engineering.
Monitoring	OpenTelemetry, Prometheus, Grafana, Loki/Jaeger	Metrics, logs and traces.

16.2 Minimum Development Hardware

Mode	Hardware
Basic development	Laptop/desktop with 16 GB RAM, Python, Docker, no GPU required for backend/RAG.
Simulation development	Dedicated GPU recommended for CARLA; 16-32 GB RAM preferred.
Perception experiments	NVIDIA GPU with 8 GB+ VRAM recommended. Cloud GPU can be used.
LLM local inference	Small models can run locally; larger models need GPU or external API.
Production demo	One CPU instance for backend + one GPU instance for simulation/model serving if available.

16.3 Local Development Services

```

services:
  frontend: Next.js dashboard
  backend: FastAPI API gateway
  worker: Celery/RQ worker for ingestion and reports
  postgres: metadata + time-series tables
  redis: cache + queue
  qdrant_or_pgvector: vector search
  minio: object storage
  prometheus: metrics
  grafana: dashboards
  vllm_or_ollama: optional local LLM serving
  carla: optional simulator service or external simulator host

```

17. Database Design

17.1 Simplified ER Diagram

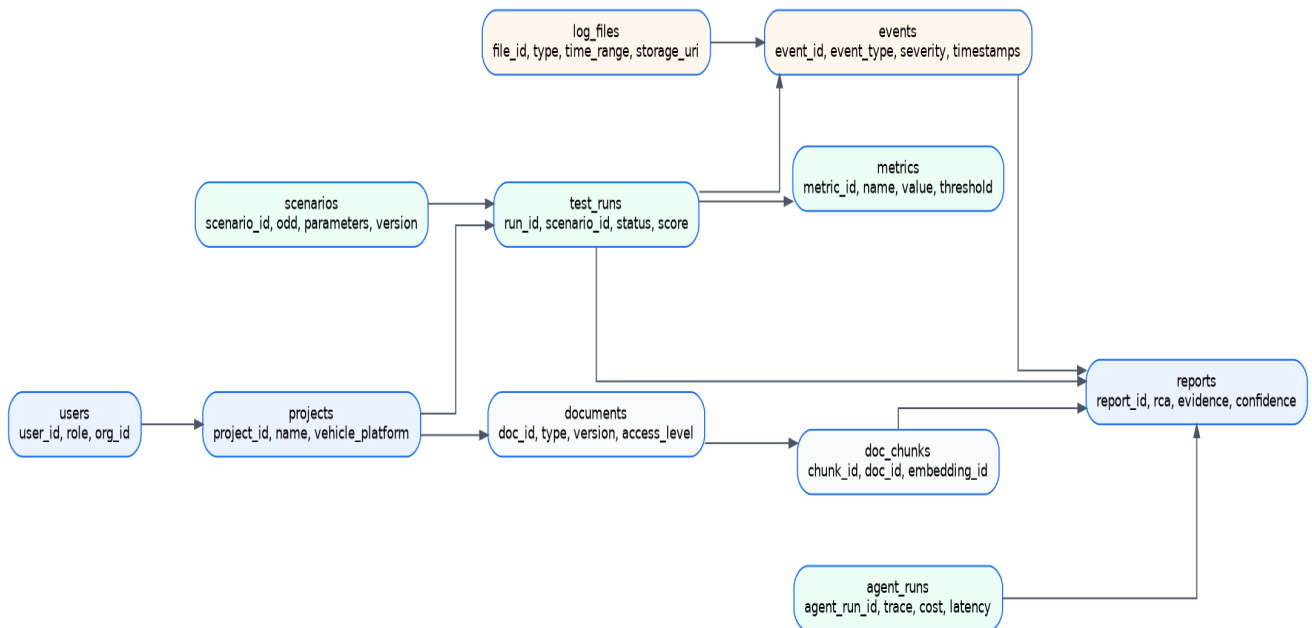


Figure 11: Simplified database model connecting users, projects, logs, scenarios, metrics, reports, documents and agent runs.

17.2 Core Tables

Table	Purpose	Important Columns
users	User accounts and roles.	user_id, org_id, role, email, created_at
projects	Vehicle platform or ADAS project workspace.	project_id, name, vehicle_platform, feature_scope
log_files	Uploaded vehicle/simulation files.	file_id, project_id, file_type, storage_uri, checksum, time_range
signals	Decoded telemetry signal definitions.	signal_id, name, unit, source, dbc_id
telemetry_windows	Event-aligned time-series slices.	window_id, event_id, start_time, end_time, features_json
events	Detected incidents or validation events.	event_id, type, severity, timestamp, source_run_id
scenarios	Scenario definitions and versions.	scenario_id, map, odd, parameters, version, status
test_runs	Executed simulation or offline validation runs.	run_id, scenario_id, model_version, result, score
metrics	Computed KPIs.	metric_id, run_id, name, value, threshold, pass_fail
documents	Engineering knowledge sources.	doc_id, type, version, access_level, storage_uri
doc_chunks	Searchable chunks for RAG.	chunk_id, doc_id, text, embedding_id, metadata
agent_runs	Agent traces and cost/latency.	agent_run_id, project_id, query, trace_json, status
reports	Generated engineering reports.	report_id, event_id, run_id, confidence, evidence_json, storage_uri
approval_tasks	Human-in-the-loop approvals.	approval_id, task_type, status, reviewer_id, decision_reason

18. API Specification

The API should be designed as a production-style backend. It should support asynchronous jobs because ingestion, simulation and report generation may take time.

18.1 Main API Endpoints

Method	Endpoint	Purpose
POST	/api/projects	Create a project workspace.
POST	/api/projects/{id}/files	Upload logs, videos, documents or DBC files.
POST	/api/ingestion/jobs	Start ingestion and indexing job.
GET	/api/events	List detected incidents and validation events.
POST	/api/query	Ask an engineering question using RAG/agents.
POST	/api/scenarios	Create or import scenario definition.
POST	/api/scenarios/{id}/run	Execute scenario in simulator.
GET	/api/runs/{id}/metrics	Fetch scenario/test metrics.
POST	/api/reports	Generate report from event/run/query.
GET	/api/reports/{id}	Download report.
GET	/api/dashboard/summary	Fetch summary KPIs.
POST	/api/approvals/{id}/decision	Approve/reject risky AI action or report claim.

18.2 Example Query API

POST /api/query

```
{
  "project_id": "proj_aeb_v1",
  "query": "Why did AEB trigger late in event EVT-2027-0042?",
  "mode": "diagnostic_agent",
  "evidence_scope": {
    "event_ids": ["EVT-2027-0042"],
    "include_documents": true,
    "include_telemetry": true,
    "include_video_frames": true
  }
}
```

Response:

```
{
  "answer": "Late AEB trigger is most likely linked to delayed object confidence crossing threshold.",
  "confidence": "medium",
  "evidence_ids": ["metric_88", "telemetry_window_42", "doc_chunk_19"],
  "unsupported_claims": [],
  "recommended_next_tests": ["Replay with lower rain intensity", "Compare perception model v2/v3"]
}
```

19. Metrics and Evaluation Criteria

19.1 Evaluation and Improvement Loop

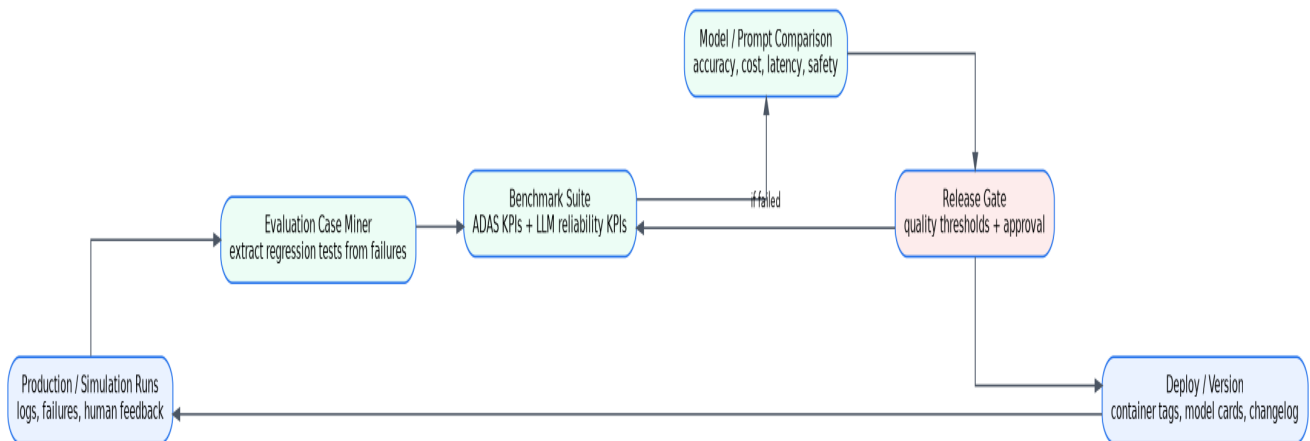


Figure 12: Failed runs become regression tests and release gates for continuous improvement.

19.2 Platform Success Metrics

Metric Group	Metrics	Target for Portfolio Demo
ADAS Safety/Performance	collision rate, min TTC, lane invasion, false negatives, braking latency	Demonstrate on 5-6 scenario types with reproducible scores.
Perception Evaluation	precision, recall, mAP, false positive/negative rate, ID switches	Run on sample/simulated dataset and show slice analysis.
Agent Reliability	task success rate, unsupported claim rate, tool error rate	Unsupported claims below 10 percent in curated test set.
RAG Quality	retrieval precision, answer faithfulness, citation accuracy	Every report claim should link to evidence.
System Performance	API latency, job duration, throughput, queue time	Dashboard shows p50/p95 latency and job status.
Cost Efficiency	tokens per report, local/API model split, cache hit rate	Model routing and caching visible in dashboard.
Reproducibility	scenario versioning, seed storage, model version tracking	Every run can be replayed from stored metadata.
Security	blocked prompt injection attempts, approval triggers, audit events	Show security test cases and audit log.

19.3 Release Gate Criteria

Gate	Condition
Data quality gate	No critical missing timestamps, corrupted files or invalid schema.
Scenario validity gate	Scenario has map, ODD, actor config, success criteria and deterministic seed.
ADAS metric gate	Critical metrics meet thresholds or are flagged as failures.
LLM evidence gate	No high-risk unsupported claims in final report.
Human approval gate	Safety-critical summaries require reviewer sign-off.
Regression gate	New version must not perform worse on locked regression scenario suite.

20. Testing Strategy

Test Type	What to Test	Tools
Unit Tests	Parsers, metrics, API schemas, route logic, signal calculations.	pytest, mypy, ruff.
Integration Tests	Upload -> ingest -> query -> report flow.	pytest, testcontainers, Docker Compose.
Simulation Tests	Scenario execution, deterministic seed, metric collection.	CARLA, Python runner.
RAG Tests	Retrieval quality, citation correctness, answer faithfulness.	Ragas/DeepEval/custom scripts.
Agent Tests	Tool call success, planning correctness, failure recovery.	LangGraph traces, mocked tools.
Security Tests	Prompt injection, unauthorized document access, unsafe tool calls.	OWASP-style test cases, custom adversarial prompts.
Performance Tests	Latency, queue time, ingestion throughput.	Locust/k6, Prometheus.
Regression Tests	Locked scenario suite and known incident queries.	CI pipeline.

20.1 Example Test Dataset Plan

- 1 Use CARLA to generate synthetic scenarios for AEB, lane keeping and cut-in testing.
- 2 Use public datasets such as nuScenes for perception and sensor-fusion experimentation where licensing permits.
- 3 Create small sample DBC-like signal definitions for demo telemetry.
- 4 Create synthetic requirements and manuals for safe public RAG demo.
- 5 Store all demo data in a separate data/demo directory with README and licenses.

21. Deployment and DevOps

21.1 Deployment Architecture

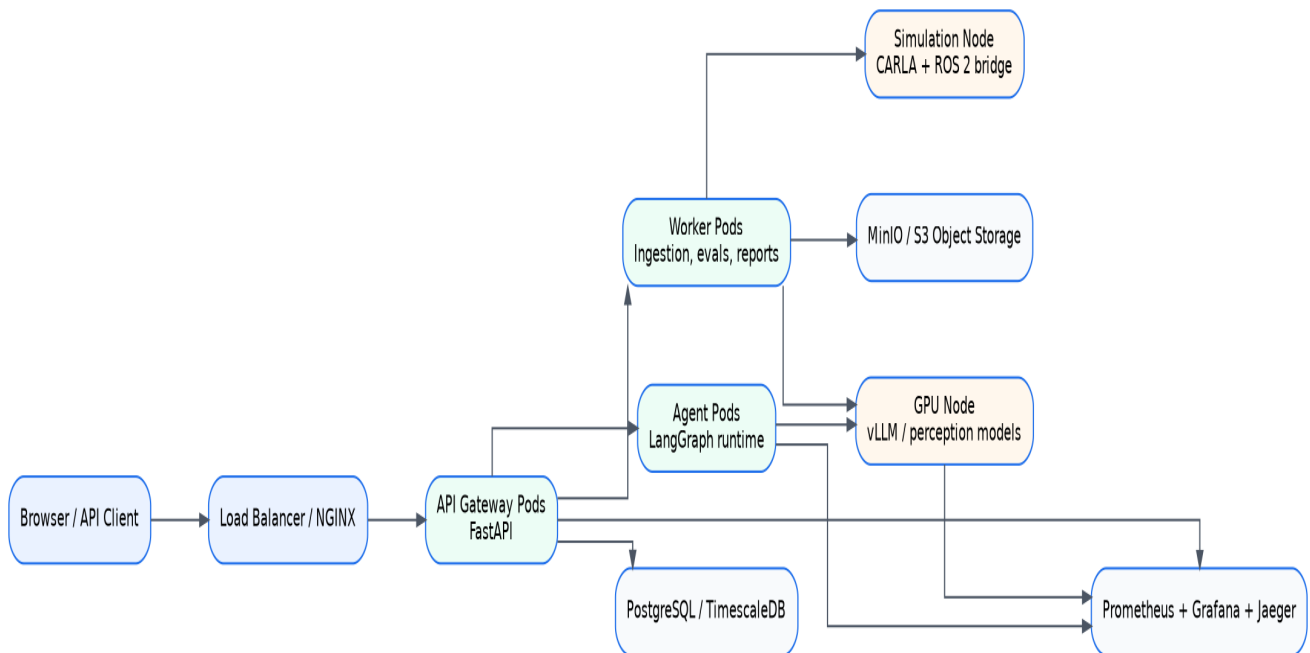


Figure 13: Production-style deployment with separate backend, workers, agents, GPU model serving, simulation and observability.

21.2 Docker Compose Services

```
version: "3.9"
services:
  frontend:
    build: ./frontend
    ports: ["3000:3000"]
  backend:
    build: ./backend
    ports: ["8000:8000"]
    env_file: .env
  worker:
    build: ./backend
    command: celery -A app.worker worker -l info
  postgres:
    image: postgres:16
  redis:
    image: redis:7
  minio:
    image: minio/minio
  qdrant:
    image: qdrant/qdrant
  prometheus:
    image: prom/prometheus
  grafana:
    image: grafana/grafana
  vllm:
    image: vllm/vllm-openai:latest
    profiles: ["gpu"]
  carla:
    image: carlasim/carla:latest
    profiles: ["simulation"]
```

21.3 CI/CD Pipeline

- 1 Run linting and type checking for backend and frontend.
- 2 Run backend unit tests and API contract tests.
- 3 Run database migration tests.
- 4 Run small RAG and agent regression suite.

- 5 Build Docker images and tag with Git commit hash.
- 6 Deploy demo environment or update staging stack.
- 7 Generate documentation and publish architecture diagrams.

21.4 Observability Dashboard

Dashboard Panel	Description
Request latency	p50/p95 API and agent response time.
Job queue	Ingestion, simulation and report jobs waiting/running/failed.
LLM usage	Token usage, model selection, cost estimate, cache hit rate.
Scenario metrics	Pass/fail rates, collision count, TTC trends, lane invasions.
Agent traces	Steps, tool calls, errors, verification results.
Security events	Prompt injection blocked, approval required, unauthorized access attempts.
Data quality	Missing timestamps, invalid signals, corrupted file count.

22. Implementation Roadmap

This roadmap is designed for a final-year student aiming for high-impact placement by 2027. The goal is to create a working product, not just documentation.

Phase	Duration	Deliverables
Phase 0: Research and setup	2 weeks	Read ADAS basics, CARLA setup, ROS 2 basics, dataset licensing, create repo and docs.
Phase 1: Backend foundation	3-4 weeks	FastAPI, auth, projects, file upload, PostgreSQL, MinIO, Docker Compose.
Phase 2: Data ingestion	4 weeks	CSV/CAN-like parser, video upload, timestamp alignment, event windows, data quality report.
Phase 3: Automotive RAG	3-4 weeks	Document parser, chunking, vector search, citations, requirement query API.
Phase 4: Metrics engine	4 weeks	TTC, lane offset, braking latency, false positives/negatives, plots and dashboards.
Phase 5: Agent diagnostics	4-6 weeks	Planner, telemetry agent, RAG agent, critic agent, report agent with trace logs.
Phase 6: Simulation integration	6-8 weeks	CARLA runner, scenario JSON schema, basic AEB/LKA scenarios, result scoring.
Phase 7: Scenario generation	4 weeks	LLM scenario authoring, parameter mutation, scenario bank, regression suite.
Phase 8: Security and approvals	3 weeks	Prompt injection tests, RBAC, human approval queue, audit logs.
Phase 9: Production polishing	4 weeks	Dashboard, PDF reports, CI/CD, documentation, demo video, benchmarks.

22.1 MVP Definition

MVP

A user can upload a synthetic ADAS log and requirement document, ask why an event failed, receive a report with metrics, citations and confidence, then run or review one simulated scenario in CARLA.

22.2 Advanced Version

- Native OpenSCENARIO 2.x inspired scenario generation and validation.
- Scenario search using optimization: find hardest cases automatically.
- Multi-modal incident report using video frames and telemetry plots.
- Perception model comparison across dataset slices.
- Automated creation of Jira issues from validated failures.
- Integration with GitHub PRs to compare ADAS behavior before and after code changes.
- Digital twin view showing vehicle, scenario, sensor outputs and AI diagnosis in one timeline.

23. GitHub Repository Structure

```
adas-intelligence-platform/
├── backend/
│   ├── app/
│   │   ├── api/           # FastAPI routes
│   │   ├── auth/         # JWT, RBAC, API keys
│   │   ├── ingestion/    # parsers, validation, sync
│   │   ├── telemetry/    # signal metrics and event windows
│   │   ├── rag/          # document indexing and retrieval
│   │   ├── agents/       # planner, telemetry, perception, critic, report agents
│   │   ├── simulation/   # CARLA/scenario runner integration
│   │   ├── evaluation/   # ADAS and LLM metrics
│   │   ├── security/     # policy, prompt injection tests, approvals
│   │   ├── reports/      # PDF/HTML reports
│   │   └── observability/ # traces, metrics, logs
│   └── tests/
├── frontend/
│   ├── app/              # Next.js pages
│   ├── components/      # UI components
│   └── dashboards/       # metrics and reports
├── simulation/
│   ├── carla_runner/
│   ├── scenarios/
│   ├── maps/
│   └── scripts/
├── data/
│   ├── demo_logs/
│   ├── demo_docs/
│   ├── demo_scenarios/
│   └── licenses/
├── docs/
│   ├── architecture.md
│   ├── api.md
│   ├── deployment.md
│   ├── evaluation.md
│   └── safety_notes.md
├── infra/
│   ├── docker-compose.yml
│   ├── prometheus.yml
│   ├── grafana/
│   └── k8s/
├── notebooks/            # experiments and analysis
├── benchmarks/          # scenario and agent benchmark suites
├── scripts/
├── README.md
└── LICENSE
```

23.1 README Must Include

- Clear problem statement and why the project matters.
- Architecture diagram and module explanation.
- Demo screenshots and video link.
- Local installation steps using Docker Compose.
- Sample data and how to run first diagnostic query.
- CARLA setup instructions and fallback mode without CARLA.
- Metrics and benchmark results.
- Security and safety disclaimers.
- Roadmap and future improvements.

24. How to Run the Project

24.1 Local Development Commands

```
# 1. Clone repository
git clone https://github.com/<username>/adas-intelligence-platform.git
cd adas-intelligence-platform

# 2. Create environment file
cp .env.example .env

# 3. Start core services
docker compose up -d postgres redis minio qdrant prometheus grafana

# 4. Start backend
cd backend
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload

# 5. Start frontend
cd ../frontend
npm install
npm run dev

# 6. Upload demo data
python scripts/upload_demo_dataset.py --project demo-aeb

# 7. Run a diagnostic query
python scripts/query.py
--project demo-aeb
--event EVT-001
--question "Why did AEB trigger late?"
```

24.2 Simulation Mode

```
# Start CARLA separately on a GPU machine if available
./CarlaUE4.sh -quality-level=Low

# Run demo scenario
python simulation/carla_runner/run_scenario.py
--scenario simulation/scenarios/aeb_cut_in_rain_001.json
--output runs/aeb_cut_in_rain_001

# Score run
python backend/scripts/score_scenario.py
--run runs/aeb_cut_in_rain_001
```

24.3 Fallback Mode Without GPU

If CARLA cannot run locally, the platform should still work using pre-recorded simulation logs and synthetic CSV telemetry. This ensures the project remains demo-ready on normal laptops.

25. Resume and Interview Positioning

25.1 Resume Project Entry

Resume Statement
Built ADAS Intelligence Platform, a production-grade AI engineering system for ADAS validation, simulation diagnostics and telemetry analysis. Designed FastAPI microservices, CARLA scenario runner, automotive RAG, multi-agent root cause analysis, time-series metrics engine, safety evidence reports, Docker deployment and observability dashboards to analyze ADAS failures with traceable evidence.

25.2 Skills Demonstrated

Skill Area	Evidence in Project
LLM/Agents	Planner, telemetry, RAG, simulation, safety critic and report agents.

Skill Area	Evidence in Project
ADAS/Automotive	AEB, ACC, LKA scenarios, CAN-like signals, TTC, lane metrics, simulation.
Systems Engineering	Microservices, workers, queues, databases, object storage, APIs.
MLOps/LLMOps	Model routing, evaluation, traces, versioning, metrics and dashboards.
Simulation	CARLA, scenario runner, reproducible runs, regression scenario bank.
Data Engineering	Multi-modal ingestion, time sync, event windows, feature extraction.
Security/Safety	RBAC, prompt injection defense, human approval, safety disclaimers.
Production Readiness	Docker, CI/CD, monitoring, documentation, benchmarks.

25.3 Interview Story

The strongest interview explanation is: "I did not try to build a self-driving car. I built the engineering intelligence layer that ADAS teams need to validate, debug and improve ADAS behavior. The system connects logs, simulation, requirements, metrics and LLM agents into a traceable workflow."

25.4 Target Roles

- AI Systems Engineer - Automotive / Robotics
- ADAS Software Engineer with AI/ML focus
- LLM/Agent Engineer for engineering tools
- Autonomous Systems Validation Engineer
- Simulation and Scenario Testing Engineer
- MLOps/LLMOps Engineer in mobility or robotics
- Data/ML Engineer for vehicle telemetry and diagnostics

26. Feature-Level ADAS Specifications

This section translates the platform into practical ADAS feature modules. Each feature should have input signals, expected outputs, metrics, failure modes and demo scenarios. This makes the project look closer to an industrial engineering tool rather than a generic AI dashboard.

26.1 Automatic Emergency Braking (AEB) Module

Item	Specification
Goal	Detect risk of collision with lead vehicle or pedestrian and evaluate whether braking was timely and appropriate.
Inputs	ego_speed, relative_distance, relative_velocity, object_class, object_confidence, brake_command, acceleration, TTC, weather, visibility.
Core Metrics	minimum TTC, braking latency, collision/no collision, false positive braking, false negative missed braking, max deceleration, jerk.
Failure Modes	late trigger, no trigger, unnecessary braking, wrong object target, sensor confidence drop, planner state blocked.
Demo Scenarios	lead vehicle sudden braking, pedestrian crossing, vehicle cut-in, ghost radar object, rain/night visibility.
Report Output	AEB timeline, risk threshold crossing, first detection time, brake command time, root cause hypothesis and confidence.

26.2 Adaptive Cruise Control (ACC) Module

Item	Specification
Goal	Maintain safe following distance and smooth speed control behind lead vehicles.
Inputs	ego_speed, lead_speed, headway_time, gap_distance, target_id, acceleration command, brake command.
Core Metrics	time headway, speed tracking error, target switch correctness, comfort jerk, min distance, cut-in response time.
Failure Modes	wrong target selection, unstable acceleration, harsh braking, target loss, delayed response to cut-in.
Demo Scenarios	cut-in vehicle, lead vehicle deceleration, stop-and-go traffic, target vehicle leaves lane.
Report Output	gap plot, speed plot, target tracking timeline, ACC state-machine summary.

26.3 Lane Keeping Assist (LKA) Module

Item	Specification
Goal	Evaluate lane detection and steering assistance to keep vehicle within lane boundaries.
Inputs	lane_boundary_left/right, lane_confidence, lateral_offset, heading_error, steering_angle, steering_torque, curvature.
Core Metrics	mean lateral offset, max lateral offset, lane invasion count, lane confidence drop, steering oscillation, curve performance.
Failure Modes	lane loss, wrong lane estimate, overcorrection, oscillation, poor curve handling, weather-induced lane confidence drop.
Demo Scenarios	curved road, faded lane marking, rain, night, construction zone, lane split/merge.
Report Output	lane overlay frames, lateral offset plot, steering response plot, confidence and failure explanation.

26.4 Traffic Sign Recognition (TSR) Module

Item	Specification
Goal	Analyze sign detection and classification performance under varying visibility and road conditions.
Inputs	camera frames, detected sign class, confidence, map/speed limit reference, lighting/weather metadata.
Core Metrics	classification accuracy, missed sign rate, wrong sign class rate, confidence calibration, detection distance.
Failure Modes	missed sign, wrong speed limit, duplicate sign, sign occlusion, low-light failure.
Demo Scenarios	night sign recognition, partially occluded sign, multiple signs, rain/fog.
Report Output	sign image evidence, recognition timeline, confidence curve and likely failure reason.

26.5 Driver Monitoring System (DMS) Optional Module

This optional module can connect with the user previous emotion/speech projects. It can detect driver attention, drowsiness or distraction and link it to ADAS events. This is a good bridge between existing computer vision experience and automotive AI.

Inputs	face landmarks, eye closure, head pose, gaze, speech/emotion features, steering activity.
Metrics	drowsiness confidence, distraction duration, attention state transitions, false alert rate.
Use Case	Explain whether driver state contributed to delayed intervention or takeover warning.
Caution	Use only demo/synthetic data for portfolio unless proper consent and privacy controls exist.

27. Dashboard and UI Specification

The dashboard should make the platform feel like a real engineering product. The UI must not be only a chat box. It should combine timeline analysis, scenario results, evidence browsing, logs, model traces and reports.

27.1 Main Screens

Screen	Purpose	Core Widgets
Project Overview	High-level project health and validation status.	Total runs, pass/fail rate, top failures, risk score, recent reports.
Incident Explorer	Inspect one ADAS event deeply.	Timeline, video frame strip, signal plots, evidence panel, AI diagnosis.
Scenario Bank	Manage generated and imported scenarios.	Scenario list, ODD filters, parameters, run history, severity score.
Simulation Runs	Track CARLA jobs and results.	Job status, map, seed, metrics, plots, replay links.
Knowledge Base	Manage documents and RAG sources.	Documents, chunks, versions, access level, retrieval preview.
Agent Trace Viewer	Debug AI workflows.	Planner steps, tool calls, retrieved evidence, verifier status, token cost.
Safety Evidence Report	Review final engineering report.	Requirements, test results, metrics, confidence, approvals, limitations.
Admin / Security	Govern users and policies.	RBAC, API keys, audit logs, prompt injection events.

27.2 Incident Explorer Layout

Top: Event summary cards
 - Event type, severity, feature, timestamp, scenario/run ID
 Middle-left: Video / frame evidence
 Middle-right: AI diagnosis and evidence citations
 Bottom: Synchronized timeline
 - speed, TTC, brake command, lane offset, detection confidence, ADAS state
 Right drawer: Requirements, DBC signal definitions, related previous incidents

27.3 Report Template

- 1 Executive summary: what failed and why it matters.
- 2 Event metadata: run ID, scenario ID, model version, map, weather, timestamp.
- 3 Evidence timeline: important moments and linked signals.
- 4 Metrics table: thresholds, observed values and pass/fail.
- 5 Root cause hypotheses: ranked with evidence and confidence.
- 6 Recommended next tests: simulations or data checks to confirm cause.
- 7 Limitations: missing data, uncertainty and required human review.
- 8 Approval section: reviewer decision and comments.

28. Data Quality and Validation Rules

Before any AI analysis starts, the platform must validate data quality. Bad timestamps or missing signals can create wrong conclusions. Data quality rules are therefore part of the core architecture.

Rule	Check	Failure Handling
Timestamp continuity	Detect gaps, duplicates and clock jumps.	Warn user, mark affected windows as low confidence.
Required signals	Verify minimum signals for selected feature.	Block metric computation if critical signals are missing.
Unit consistency	Check speed, distance, acceleration and angle units.	Normalize units and record conversion metadata.
File integrity	Checksum, file size and parser validation.	Reject corrupted files and show reason.
Scenario completeness	Map, weather, actors, ego state and success criteria exist.	Prevent scenario execution until complete.
Ground truth quality	Check label coverage and annotation schema.	Mark perception metrics as approximate if labels are incomplete.
Document freshness	Check version, valid date and access level.	Prefer latest trusted document; show older-source warning.
Evidence sufficiency	Check if generated conclusion has enough supporting data.	Downgrade confidence or ask for more data.

28.1 Confidence Penalty Rules

Condition	Confidence Impact
Critical signal missing	Set report confidence to Low or Blocked.
Only one evidence source supports claim	Maximum confidence Medium.
Contradictory evidence exists	Require human review and list contradiction.
Simulation-only evidence	Do not claim real-world validation.
Outdated document used	Add warning and reduce confidence.
LLM produced unsupported claim	Remove claim or mark unsupported.

29. Motorsport / F1 Extension

The same platform architecture can be adapted for motorsport engineering. Motorsport teams care about telemetry, simulations, strategy, vehicle behavior, driver feedback and rapid decision-making. The project can include a future extension called Race Engineering Intelligence Layer.

Motorsport Need	AIP Extension
Telemetry analysis	Analyze lap speed, throttle, brake pressure, steering angle, tire temperature and sector timing.
Setup comparison	Compare suspension, aero or tire setup changes across sessions.
Race strategy	Use agents to reason over pit windows, tire degradation and safety car scenarios.
Driver coaching	Summarize where time was lost using telemetry overlays and natural-language explanations.
Reliability monitoring	Detect anomalies in temperatures, pressures, vibrations and power unit behavior.
Simulation correlation	Compare simulator prediction with actual telemetry and flag mismatch.

Career Angle

This extension helps connect the project to F1, Le Mans, racing teams and high-performance automotive roles without pretending to have proprietary race data.

30. Detailed Weekly Execution Plan

Week	Engineering Goal	Visible Output
1	Repo setup, architecture docs, Docker skeleton.	README, diagrams, docker-compose booting.
2	FastAPI projects, uploads, PostgreSQL schema.	Upload demo document/log file.
3	Document ingestion and RAG.	Ask questions over requirements with citations.
4	Telemetry CSV ingestion and signal windows.	Plot speed/TTC/brake timeline.
5	AEB metrics engine.	AEB late trigger report from synthetic data.
6	Agent planner + telemetry agent.	Agent trace with tool calls.
7	Verifier + report generator.	PDF/HTML evidence report.
8	Dashboard MVP.	Incident explorer page.
9	CARLA setup and first scenario.	Run lead vehicle braking scenario.
10	Scenario scoring and scenario bank.	Pass/fail metrics stored.
11	Scenario generation and mutation.	Generate 20 variations of cut-in scenario.
12	Security, approvals, audit logs.	Prompt injection test and approval queue.
13	Benchmarks and regression suite.	Metrics report comparing versions.
14	Polish, demo video and deployment.	Portfolio-ready GitHub project.

31. Risk Register

Risk	Impact	Mitigation
CARLA setup is heavy	Slows implementation on normal laptop.	Support fallback with pre-recorded logs; use cloud/GPU only for demos.
Real vehicle data unavailable	Limits realism.	Use public datasets, synthetic logs and simulation data.
Project becomes too broad	Difficult to finish.	Build MVP around AEB late braking diagnosis first.
LLM hallucinations	Unsafe or incorrect reports.	Evidence verification, confidence levels and human approval.
Public dataset licensing issues	Cannot redistribute data.	Provide scripts and links; include license notes.
Simulation not equal to reality	Overclaiming project value.	State limitations clearly and focus on validation workflow.
Complex tech stack	Integration burden.	Start with Docker Compose and modular milestones.

26.1 MVP Cut Strategy

If time becomes limited, reduce scope to one feature: AEB late braking diagnostics. Build ingestion, metrics, RAG, one agent workflow, one scenario type and one dashboard deeply. A deep single workflow is better than many shallow unfinished modules.

32. References and Source Notes

The following references informed the project specification. These are not copied as implementation code; they are used to align the project with common automotive, simulation, robotics and LLM security concepts.

Reference	Relevance	URL
CARLA Simulator	CARLA is an open-source simulator developed to support development, training and validation of autonomous driving systems.	https://carla.org/
CARLA Documentation	CARLA is described as a modular and flexible autonomous driving simulator.	https://carla.readthedocs.io/en/latest/start_introduction/
ASAM OpenSCENARIO DSL	Designed for large-scale verification and validation of AV/ADAS functionality and reusable scenario descriptions.	https://www.asam.net/standards/detail/openscenario-dsl/
ASAM OpenDRIVE	Provides road network descriptions that can be used in simulations for ADAS/AD development and validation.	https://www.asam.net/standards/detail/opendrive/
ROS 2 Documentation	ROS 2 documentation covers nodes, interfaces, topics, services, actions and tools used in robotics systems.	https://docs.ros.org/en/humble/index.html
AUTOSAR Adaptive Platform	Implements the AUTOSAR Runtime for Adaptive Applications and targets high-performance automotive ECUs.	https://www.autosar.org/standards/adaptive-platform
nuScenes Dataset	Public large-scale autonomous driving dataset with sensor suite data for urban driving research.	https://www.nuscenes.org/
nuPlan Dataset	Large-scale planning dataset with many hours of human driving data.	https://www.nuscenes.org/nuplan
OWASP LLM Top 10	Lists major LLM application risks including prompt injection, sensitive information disclosure and excessive agency.	https://owasp.org/www-project-top-10-for-large-language-model-applications/

Reference	Relevance	URL
OWASP Prompt Injection	Defines prompt injection as user prompts altering model behavior or output in unintended ways.	https://genai.owasp.org/llmrisk/llm01-prompt-injection/
ISO 26262 overview - Synopsys	Overview of functional safety process and ASIL risk classification for automotive E/E systems.	https://www.synopsys.com/glossary/what-is-iso-26262.html
SOTIF / ISO 21448 overview - ASAM	SOTIF relates to absence of unreasonable risk due to hazards from intended functionality limitations.	https://report.asam.net/iso-21448-sotif

33. Final Conclusion

The ADAS Intelligence Platform is intentionally designed as a high-impact, production-grade portfolio project. It connects AI systems, LLM agents, automotive simulation, telemetry analytics, RAG, validation metrics and safety evidence. It can help the developer position himself for both top AI companies and automotive/motorsport engineering roles.

Final Positioning

This project says: "I can build AI infrastructure for complex engineering systems." That is stronger than saying: "I trained another object detection model."